ReqMind AI — Analyseur Intelligent d'Exigences Logicielles

Guide Pédagogique Intégral : Architecture, Modèles, Heuristiques & Résultats

Nom et prénom : **Zakariae CHELLE** Groupe : **5IASDT G1**

Module : **Traitement Automatique du Langage Naturel (TALN)**

Ce document constitue la synthèse pédagogique complète du projet ReqMind AI. Conçu pour expliciter chaque détail conceptuel, mathématique et logiciel, il détaille l'ingénierie des exigences par le TALN, la comparaison empirique des trois classifieurs (SVM baseline, SVM optimisé calibré et DeBERTa-v3), les algorithmes d'analyse qualité (IEEE 830), la détection de quasi-doublons sémantiques avec gardes de sécurité, la gouvernance Human-in-the-Loop et les résultats mesurés.

Partie 1 — Contexte Industriel & Problématique de l'Ingénierie des Exigences

L'Ingénierie des Exigences (Requirements Engineering - RE) constitue la pierre angulaire de tout cycle de vie logiciel. Selon les études empiriques de l'IEEE et du Standish Group, plus de 70% des défaillances critiques et des dépassements budgétaires en production trouvent leur origine dans des spécifications d'exigences (SRS) incomplètes, ambiguës, mal classifiées ou redondantes.

1.1 La distinction fondamentale : Exigences Fonctionnelles (FR) vs Non-Fonctionnelles (NFR)

- **Exigences Fonctionnelles (FR) :** Elles décrivent les actions, comportements, services et transformations de données que le logiciel doit accomplir en réponse à des entrées spécifiques (ex. : *« Le système doit permettre aux clients d'ajouter un article au panier »*).
- **Exigences Non-Fonctionnelles (NFR) :** Elles définissent les critères de qualité, contraintes opérationnelles, seuils de performance, exigences de sécurité et de conformité réglementaire régissant le comportement global du système (ex. : *« Le temps de réponse de l'API de paiement ne doit pas dépasser 500 ms à pleine charge »*).

> ***Pourquoi cette distinction est critique :*** *Dans les projets agiles et les cahiers des charges industriels, les ingénieurs mélangent fréquemment les préoccupations fonctionnelles et qualitatives dans une même phrase. Ce flou nuit à la conception architecturale : un développeur concevra une simple fonctionnalité là où un architecte réseau doit déployer une infrastructure à haute disponibilité.*

1.2 Les trois fléaux des spécifications textuelles

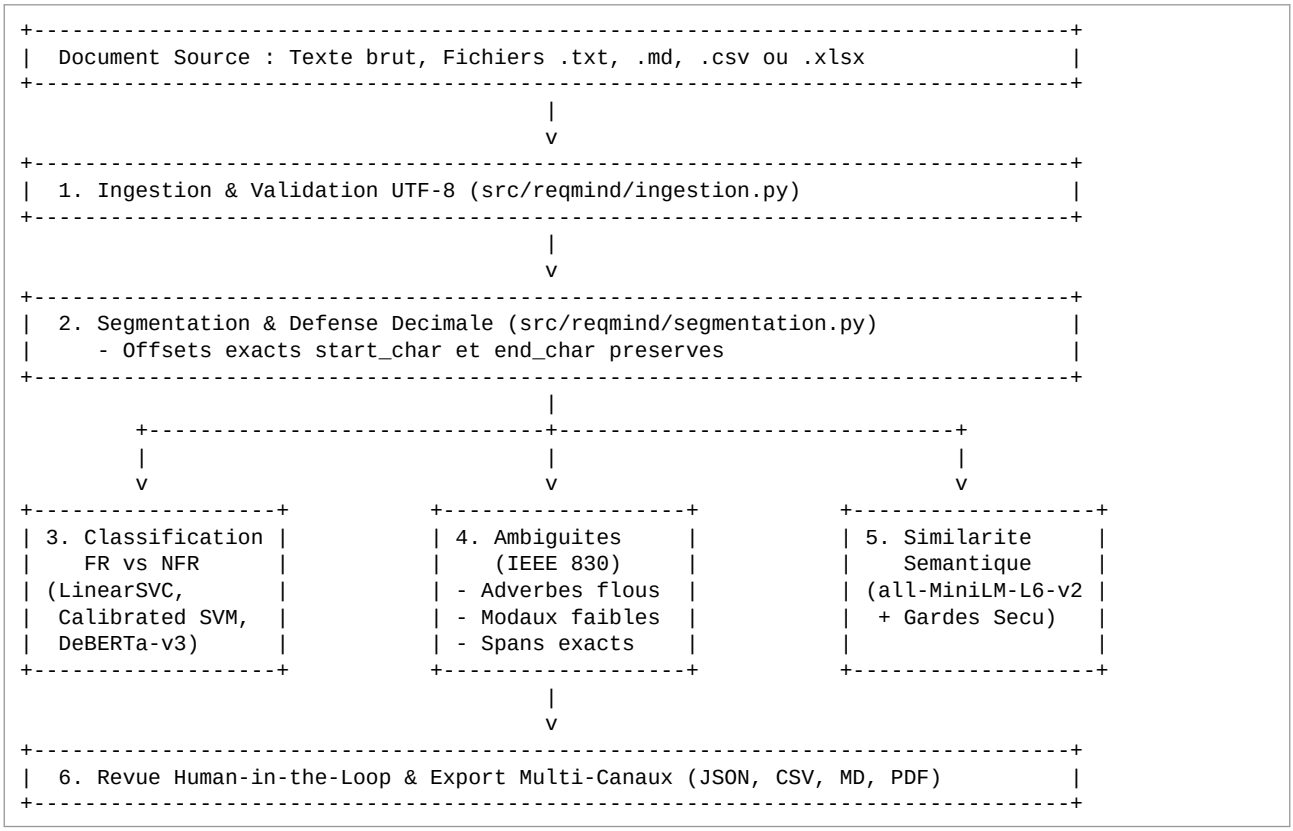
1. La mauvaise classification taxonomique : Des contraintes vitales de sécurité ou de latence sont traitées comme de simples tickets de fonctionnalités et sous-estimées dans les sprints.

2. L'ambiguïté lexicale et syntaxique (Norme IEEE 830) : Présence d'adverbes vagues (« rapidement », « efficacement ») et de modaux non-contraignants (« devrait », « pourrait ») rendant l'exigence impossible à vérifier par un test automatique d'acceptation.

3. La redondance et les quasi-doublons contradictoires : Dans les projets complexes impliquant de multiples équipes, des exigences similaires sont rédigées avec des formulations divergentes ou des valeurs de SLA incompatibles.

Partie 2 — Architecture Technique & Pipeline de Traitement

ReqMind AI repose sur un pipeline séquentiel hautement modulaire encapsulé dans le package Python src/reqmind/. L'orchestrateur central coordonne les étapes d'ingestion, de segmentation, d'analyses NLP en parallèle, de revue humaine et d'export multi-canaux.



2.2 La segmentation à défense décimale et conservation des offsets

Une erreur classique en TALN consiste à découper aveuglément par les points. L'exigence *« The service shall maintain 99.9% availability during business hours. »* serait alors découpée à tort à cause du point décimal. ReqMind AI protège les motifs numériques \d+\.\d+ et les numérotations hiérarchiques (1.1, 1.2), tout en enregistrant start_char et end_char pour chaque élément.

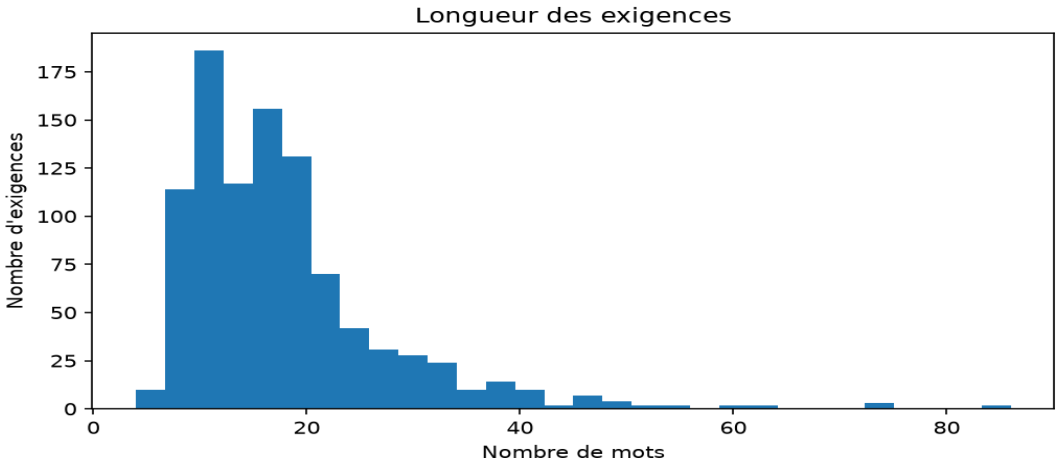


Figure 1 — Distribution des longueurs de textes des exigences en nombre de mots.

Partie 3 — Classification FR vs NFR : Étude Approfondie des Modèles

ReqMind AI intègre et compare trois architectures de classification :

- **Modèle 1 (Baseline TF-IDF + LinearSVC) :** Unigrammes et bigrammes avec pondération sous-linéaire et LinearSVC (C=1.0). Inférence ultra-rapide (< 0.5 ms), mais aveugle aux dépendances à longue distance.
- **Modèle 2 (SVM Optimisé & Calibré) :** FeatureUnion combinant n-grammes de mots (1, 2) et n-grammes de caractères (3, 5), pondération équilibrée (class_weight='balanced') et probabilités calibrées via le scaling de Platt (CalibratedClassifierCV).
- **Modèle 3 (DeBERTa-v3-small Fine-Tuned) :** Architecture Transformer avec attention désenchevêtrée (Disentangled Attention) décomposant l'attention entre contenu et position relative. Régularisée par Label Smoothing (0.1).

3.2 Résultats expérimentaux sur le Benchmark Humain Annoté (60 Exemples)

Métrique d'Évaluation	Baseline SVM	SVM Calibré	DeBERTa-v3-small
Exactitude (Accuracy)	76.67% (46/60)	80.00% (48/60)	96.67% (58/60)
Macro-F1	76.64%	79.80%	96.67%
Précision FR	75.00%	75.00%	96.67%
Rappel FR	80.00%	90.00%	96.67%
F1-Score FR	77.42%	81.82%	96.67%
Précision NFR	78.57%	87.50%	96.67%
Rappel NFR	73.33%	70.00%	96.67%
F1-Score NFR	75.86%	77.78%	96.67%
Nombre d'Erreurs Global	14 erreurs	12 erreurs	2 erreurs (-85.7%)
Latence CPU / Exigence	0.46 ms	0.35 ms	14.66 ms (~42x)

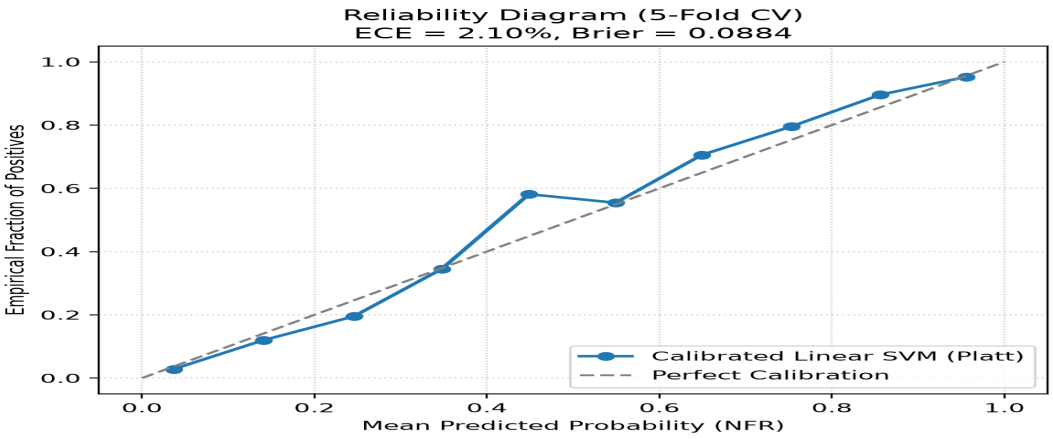


Figure 2 — Courbe de calibration du SVM optimisé après scaling de Platt.

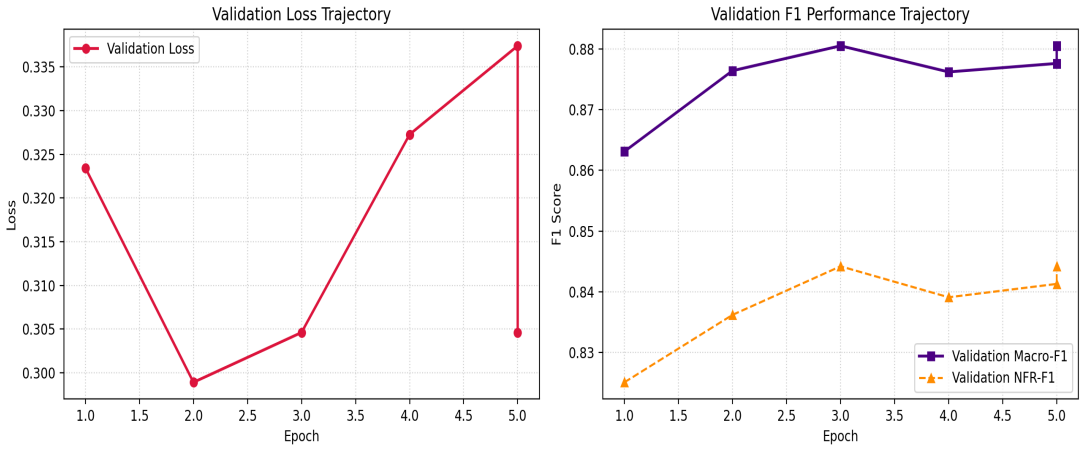


Figure 3 — Courbes d'apprentissage de DeBERTa-v3-small (Convergence perte et Macro-F1).

3.4 Pourquoi DeBERTa surpasse les modèles classiques

Sur l'exigence : « *The system shall encrypt all sensitive customer data before writing to the database.* », le SVM prédit **FR** à tort, trompé par les verbes d'action (*shall, write, database*). DeBERTa prédit **NFR** avec 96% de confiance car son attention désenchevêtrée comprend que *encrypt* est une contrainte de sécurité appliquée à l'opération et non une fonction métier offerte à l'utilisateur.

Partie 4 — Détection d'Ambiguïtés & Défauts Qualité selon IEEE 830

La norme IEEE 830 exige que toute exigence soit vérifiable et testable. ReqMind AI analyse les textes à travers quatre classes de défauts :

Catégorie	Mots Détectés	Risque Industriel	Recommandation
Adverbes Vagues	quickly, efficiently, regularly	Test impossible à automatiser.	Remplacer par un SLA mesurable.
Revendications Subjectives	user-friendly, intuitive	Appréciation subjective.	Remplacer par un score d'utilisabilité.
Modaux Non-Contraignants	should, could, may, might	Viole la RFC 2119 (facultatif).	Employer obligatoirement 'shall'.
Proportionnalité Faible	as appropriate, if feasible	Échappatoire technique floue.	Définir les conditions logiques exactes.

Gestion des Faux-Positifs (Dismiss) : Si un terme technique légitime est détecté (ex. : « *fast Fourier transform* »), l'ingénieur clique sur 'Rejeter l'alerte' dans l'interface. L'alerte est désactivée dans l'audit sans modifier le texte source.

Partie 5 — Détection Sémantique de Doublons & Gardes de Sécurité

ReqMind AI utilise le bi-encodeur dense sentence-transformers/all-MiniLM-L6-v2 (384 dimensions) pour calculer la similarité cosinus entre exigences. Pour éviter les erreurs grossières des approches vectorielles pures, le système introduit deux gardes logiques :

- **La Garde de Négation :** Alerte si deux exigences partageant une forte similarité syntaxique ont une polarité inversée (ex. : *shall allow* vs *shall not allow*).
- **La Garde de Conflit SLA / Numérique :** Empêche de fusionner deux exigences partageant la même formulation mais avec des seuils numériques divergents (ex. : « *complete within 2 hours* » vs « *complete within 6 hours* »).

Partie 6 — Gouvernance Human-in-the-Loop & Traçabilité Multi-Canaux

Machine à états finis de révision : Chaque exigence suit le cycle PENDING -> APPROVED / EDITED / REJECTED. Le champ predicted_label reste immuable pour garantir l'auditabilité, tandis que l'adjudication de l'ingénieur est enregistrée dans reviewer_label.

Quatre canaux d'exportation :

1. **JSON Intégral :** Sérialisation Pydantic complète avec offsets exacts et validateur aller-retour.
2. **CSV Tabulaire :** Feuille d'audit avec labels résolus pour import Jira/DOORS.
3. **Markdown Exécutif :** Synthèse lisible et distributions globales.
4. **PDF Haute Fidélité :** Document ReportLab avec canvas numéroté (« Page X / Y ») et cartes KPI.

Partie 7 — Guide de Démonstration en 5 Minutes pour Soutenance

Minute 1 (Ingestion) : Charger le scénario 'E-Commerce Platform SRS'. Montrer la segmentation propre sans rupture des décimales (ex: '99.9% uptime').

Minute 2 (Qualité IEEE 830) : Pointer les badges en surbrillance sur 'quickly' et 'user-friendly'. Démontrer le bouton de rejet de faux positif.

Minute 3 (Doublons & SLA) : Afficher les paires à similarité > 0.85 et mettre en avant l'alerte de conflit numérique SLA.

Minute 4 (Revue & Export) : Basculer le statut en APPROVED/EDITED et télécharger le rapport d'audit PDF complet.

Minute 5 (Laboratoire d'Évaluation) : Dans l'Onglet 8, lancer la comparaison SVM vs DeBERTa sur le test set de 60 cas. Mettre en valeur le bond d'exactitude (+20 pts : 76.7% -> 96.7%) et la matrice de confusion interactive.